

Travaux Pratiques : Jeu de Chasseur et Bonhomme

Énoncé

Il s'agit d'un jeu à deux joueurs : un bonhomme et un chasseur, qui jouent à tour de rôle. Le bonhomme est contrôlé via la console, tandis que le chasseur est dirigé par le programme.

- Les deux joueurs se déplacent sur une grille composée de cellules. Chaque cellule peut contenir des éléments influençant le niveau d'énergie des joueurs : certains objets augmentent l'énergie (objets bonus), d'autres la diminuent (objets malus). Contrairement aux malus, les bonus ne sont pas renouvelables. Les pertes ou gains des énergies ne sont pas connus par les joueurs. Par contre, ils connaissent la nature des objets (bonus ou malus).
- L'énergie d'un joueur diminue d'une unité à chaque déplacement.
- La grille contient également une cible, dont la position est connue que par le chasseur.
- Chaque joueur commence avec une énergie initiale et ne peut voir que les cases adjacentes à sa position actuelle.
- Le chasseur se déplace toujours vers la case adjacente la plus proche de la cible, sauf si le bonhomme est dans une case adjacente. Dans ce dernier cas, il l'élimine car l'objectif principal du chasseur est d'éliminer le bonhomme pour s'accaparer de la cible.
- Le bonhomme peut choisir librement parmi les cases adjacentes.
- Le jeu se termine lorsqu'un joueur atteint la cible ou que l'énergie des deux est complètement épuisée.

Questions

1. Donnez le diagramme de classes de ce jeu. On suppose que la grille est composée de cellules (cases). À chaque objet `Cellule`, au plus un objet de la classe `Element` est associé. Un objet `Element` est associé à un seul objet `Cellule`. La classe `Element` dispose de la méthode `getEnergie()`, qui donne l'énergie gagnée ou perdue par un joueur lorsque celui-ci traverse la cellule contenant l'élément.
2. Donnez le diagramme de séquence de la fonctionnalité `seDeplacer` d'un joueur.
3. Donnez une implémentation Java de cette version minimale de ce jeu.
4. On souhaite maintenant ajouter de nouvelles classes que l'on ne peut pas modifier, et qui sont similaires à la classe `Element`, à l'exception du fait que la méthode permettant d'obtenir l'énergie porte un nom différent.

Exemples :

- `Feu → int energieMoins()`

- `Predateur → int perteEnergie()`
- `Fruits → int gainEnergie()`

Il est important de noter que la classe `Cellule` ne doit pas être modifiée lors de l'ajout d'une nouvelle classe. Or, dans la solution précédente, toute extension nécessitait une modification soit de la nouvelle classe, soit de la classe `Cellule`, ce qui viole "fermeture à la modification/ Ouverture à l'extension". Pour résoudre ce problème, nous appliquons le patron de conception `Adaptateur`, décrit dans la Figure 1 à renommage près :

- Créer une interface Java `ElementInterface` contenant la signature de la méthode `getEnergie()`.
- Ajouter dans la classe `Cellule` un attribut de type `ElementInterface`.
- Pour chaque nouvelle classe (par exemple `Feu`), créer une classe adaptatrice (par exemple `AdaptateurFeu`) qui :
 - implémente l'interface `ElementInterface`,
 - utilise une instance de la classe à adapter (`Feu`),
 - redéfinit la méthode `getEnergie()` en appelant la méthode spécifique de la classe adaptée (`energieMoins()`).

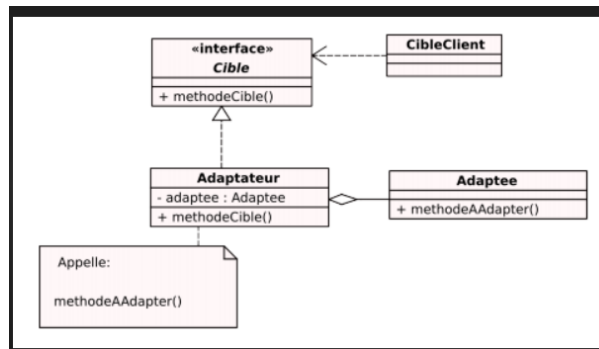


FIGURE 1 – Schéma du patron de conception adaptateur.

5. Dans la question précédente, vous venez d'implémenter le patron de conception adaptateur. Expliquez l'intérêt de ce patron.
6. On suppose que le bonhomme dispose d'un papier pour noter toutes les cellules qu'il a traversées, avec l'information sur les bonus ou malus. La stratégie initiale de déplacement du bonhomme est présentée ci-dessus. Lorsque son énergie dépasse une constante, `highValue` (sauvegardée dans une interface Java), il est autorisé à changer de stratégie de déplacement : il peut choisir n'importe quelle case qu'il n'a pas encore visitée. Rappelons que, contrairement au chasseur, le bonhomme ne connaît pas la cible à atteindre. Dès que l'énergie du bonhomme descend sous le seuil `highValue`, il est contraint de se déplacer selon sa stratégie initiale. Pour cela, il vous est demandé d'implémenter le patron de conception stratégie, décrit dans la Figure 2 à renommage près :
7. Expliquer pourquoi, vous pouvez ajouter de nouvelles stratégies sans modifier la classe `Joueur`, juste en créant une nouvelle classe qui implémente `DeplacementStrategy`. Conclure.
8. Compléter la description du jeu afin d'implémenter le pattern `Observer`.
9. Compléter la description du jeu afin d'implémenter d'autres pattern de conception.

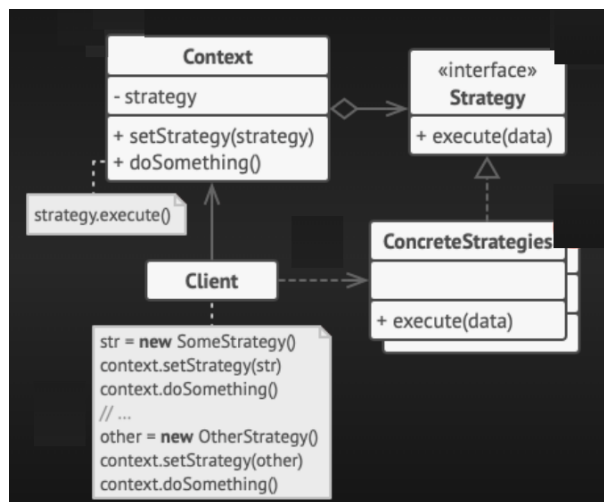


FIGURE 2 – Schéma du patron de conception stratégie.